

Problem Solving Session (CS648)

Group ID: 230305

Members: Kartikey Singh Tomar (240529), Daksh Saijwal (240318)
Om Ji Gupta (230719), Chaitanya Bramhapurikar (230305)

14 February 2026

Problem 1. There is a bag containing n balls numbered from 1 to n . We do not know n . Design a monte-carlo algorithm to estimate n such that given $\epsilon > 0$ and $c > 0$,

$$(1 - \epsilon)n \leq n' \leq (1 + \epsilon)n$$

with probability at least c .

The number of samplings we require so that our prediction (some function of those samplings) lies in the said range will be some function of ϵ and c .

Approach 1: Just return the maximum number obtained till now

Let us assume random variable X_i denotes the maximum of the first i samples. We aim to find the expected behavior of X_i to estimate n . Also let random variable t_i denote the value of the i -th sample. The probability that the maximum of the sampled values, X_i , is greater than some value t is:

$$\begin{aligned} P(X_i > t) &= P\left(\bigcup t_i > t\right) \\ &= 1 - \prod_i P(t_i \leq t) (\because \text{individual samples are all independent}) \\ &= 1 - \left(\frac{t}{n}\right)^i \end{aligned}$$

Conversely, the probability that the maximum is less than t is:

$$P(X_i < t) = \left(\frac{t-1}{n}\right)^i$$

We define our estimator as $n' = X_i$ (the maximum value observed so far). We desire $n' > (1 - \epsilon)n$ (note that $n' < (1 + \epsilon)n$ is trivially true, as we are taking maximum across all the samples):

$$P(X_i < (1 - \epsilon)n) = \left[\frac{(1 - \epsilon)n - 1}{n} \right]^i \leq (1 - \epsilon)^i$$

We require $P(X_i \geq (1 - \epsilon)n) \geq c$ which is equivalent to $P(X_i < (1 - \epsilon)n) \leq 1 - c$. Therefore we desire

$$(1 - \epsilon)^i \leq 1 - c \iff i \geq \log_{1-\epsilon}(1 - c)$$

We conclude that if we sample at least $\log_{1-\epsilon}(1 - c)$ times and take their maximum, then we are sure that that maximum will lie in the said range with probability at least c .

Our next question was: Can we do better?

Next try: Mean-Based Estimation

Note: Unfortunately, in the actual problem solving session we accidentally wrote the wrong Chebyshev's inequality and hence we arrived at an incorrect result. We have fixed the mistakes below.

Algorithm: Sample t values $x_1, x_2, \dots, x_t$. Calculate the estimate $\hat{n}$ as:

$$\hat{n} = 2 \left(\frac{\sum_{i=1}^t x_i}{t} \right) - 1$$

Analysis: First of all,

$$E[\hat{n}] = \frac{2}{t} \cdot t \cdot \left(\frac{n+1}{2} \right) - 1 = n$$

Next, note that if we sample “more” number of times, our estimator $\hat{n}$ will eventually get closer and closer to n . So we calculate the variance of $\hat{n}$ to get a idea of how many samples will suffice.

$$\begin{aligned} \sigma^2 &= \text{Var}(\hat{n}) \\ &= \text{Var} \left(\frac{2}{t} \sum_{i=1}^t x_i - 1 \right) \\ &= \frac{4}{t^2} \text{Var} \left(\sum_{i=1}^t x_i \right) \quad (\text{Since constants do not affect variance}) \end{aligned}$$

Since the samples x_i are independent, the variance of the sum is the sum of the variances:

$$\text{Var} \left(\sum x_i \right) = \sum \text{Var}(x_i) = t \cdot \text{Var}(x)$$

For a uniform distribution from 1 to n , $\text{Var}(x) \approx \frac{n^2}{12}$. Substituting this back:

$$\sigma^2 \approx \frac{4}{t^2} \cdot \left(t \cdot \frac{n^2}{12} \right) = \frac{n^2}{3t}$$

We want the error to be bounded by ϵn with probability at least c :

$$P(|\hat{n} - n| \leq \epsilon n) \geq c \iff P(|\hat{n} - n| \geq \epsilon n) \leq 1 - c$$

Using Chebyshev's inequality $P(|x - \mu| \geq k) \leq \frac{\sigma^2}{k^2}$, we set $k = \epsilon n$:

$$\frac{\sigma^2}{(\epsilon n)^2} \leq 1 - c$$

Substituting $\sigma^2 \approx \frac{n^2}{3t}$:

$$\begin{aligned} \frac{n^2/3t}{\epsilon^2 n^2} &\leq 1 - c \\ \frac{1}{3t\epsilon^2} &\leq 1 - c \\ t &\geq \frac{1}{3\epsilon^2(1 - c)} \end{aligned}$$

The number of required samples t required depends only on ϵ and c , and is independent of n .

Note: We tried putting different values of ϵ and c (we did this **after** the session) and our earlier approach (taking the maximum) seems to be better than this one (see table below):

c	ϵ	Max	Mean
0.90	0.10	22	334
0.90	0.05	45	1,334
0.90	0.01	230	33,334
0.95	0.10	29	667
0.95	0.05	59	2,667
0.95	0.01	299	66,667
0.99	0.10	44	3,334
0.99	0.05	90	13,334
0.99	0.01	459	333,334

We pondered over this question: Can we establish some theoretical bound on estimates which would *definitely* outperform all? (More like, what's the *best* way to estimate?)

Next Try: Can we use Chernoff Bound somehow?

Well, we did try this as mentioned in our work too, but pursuing it further after the session didn't lead to anything significant, so we decide to omit it altogether.

Problem 2. Design and analyze the query algorithm for 5-approximate distance oracle.

Attempt to Extend the In-Class Distance Oracle to a 5-Approximation

In this problem, we attempt to analyze and extend the distance oracle construction discussed in class to obtain a 5-approximate distance oracle. We will present our approach here and the way in which we tried to derive it.

Notation and Objective

For any two vertices $u, v \in V$, we use the following notation:

- $\delta(u, v)$ denotes the *reported distance* between u and v .
- $\bar{\delta}(u, v)$ denotes the *actual shortest path* returned by a distance oracle.

We assume access to a 3-approximate distance oracle discussed in class, which guarantees that for all $u, v \in V$,

$$\bar{\delta}(u, v) \leq \delta(u, v) \leq 3 \bar{\delta}(u, v).$$

Our aim in this problem is to design and analyze a query algorithm that reports distances $\delta(u, v)$ such that

$$\bar{\delta}(u, v) \leq \delta(u, v) \leq 5 \bar{\delta}(u, v),$$

by extending the in-class construction using additional sampling and case analysis.

NOTE: during the actual problem solving session, we confused between the notation a little bit, and used $\text{Ball}(u, V, L_1)$ instead of $\text{Ball}(u, L_1, L_2)$. We've corrected it here. Apologies for the inconvenience.

Preprocessing and Setup

Fix a vertex $u \in V$. All vertices are arranged in increasing order of their distance from u .

We construct two landmark sets using random sampling as follows. First, each vertex in V is sampled independently with probability p_1 to form a subset S_1 . Then, each vertex in S_1 is sampled independently with probability p_2 to form a subset S_2 . The probabilities p_1 and p_2 will be fixed later.

From these sets, we define:

- L_1 as the vertex in S_1 with minimum distance from u ,
- L_2 as the vertex in S_2 with minimum distance from u .

We define balls centered at u as follows:

$$\text{Ball}(u, V, L_i) = \{v \in V \mid \delta(u, v) < \delta(u, L_i)\}.$$

During preprocessing, the following information is stored:

- For each vertex $u \in V \setminus \{L_1\}$, we store the distances from u to all vertices in $\text{Ball}(u, V, L_1)$.
- For each vertex $u \in V$, we store the set of vertices from the first landmark set that lie inside the $\text{Ball}(u, L_1, L_2)$.
- For each vertex in L_2 , we store the distance from it to every vertex in V .

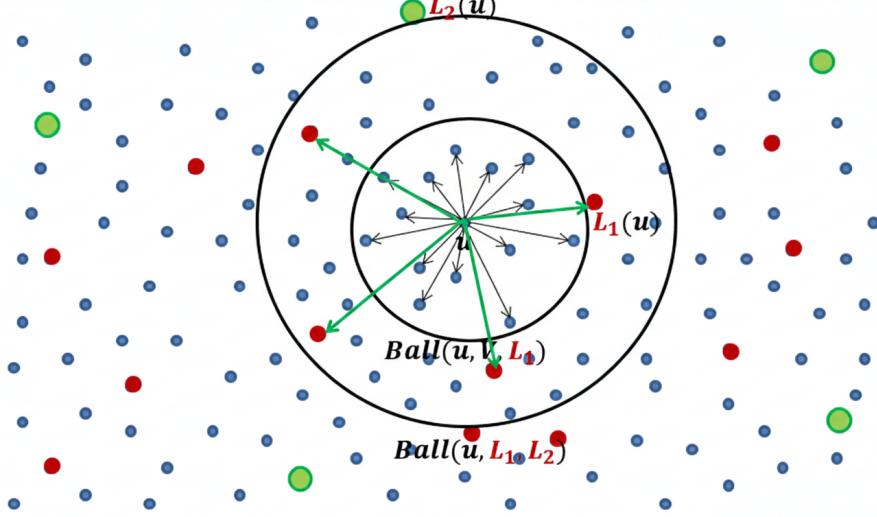


Figure 1: Illustration of the ball-based landmark structure around a fixed vertex u . The inner $\text{Ball}(u, V, L_1)$ contains vertices whose actual distance $\bar{\delta}(u, v)$ is smaller than $\bar{\delta}(u, L_1)$, while the outer $\text{Ball}(u, L_1, L_2)$ is defined similarly using L_2 . The landmark vertices L_1 and L_2 are chosen as the closest sampled vertices to u from the sets S_1 and S_2 , respectively. Different query cases correspond to whether the queried vertex lies inside the inner ball, between the two balls, or outside the outer ball.

Query Algorithm

Given a query vertex v_k , we report the distance based on its position relative to the above balls.

Case 1: $v_k \in \text{Ball}(u, V, L_1)$

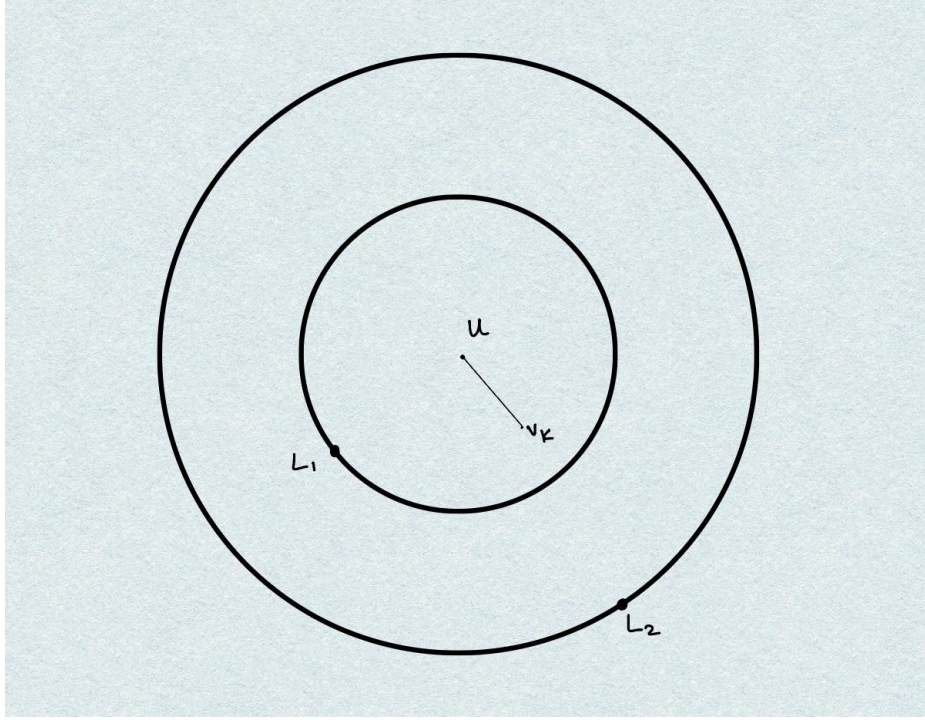
Since $v_k \in \text{Ball}(u, V, L_1)$, the distance between u and v_k is directly reported, as $\text{Ball}(u, V, L_1)$ stores distance data for all vertices inside it. Hence,

$$\delta(u, v_k) = \bar{\delta}(u, v_k).$$

Therefore,

$$\delta(u, v_k) \leq 5 \bar{\delta}(u, v_k),$$

which is trivially satisfied.



Case 2: $v_k \notin \text{Ball}(u, L_1, L_2)$

In this case, the reported distance is given by routing through the landmarks:

$$\delta(u, v_k) = \bar{\delta}(u, L_1) + \bar{\delta}(L_1, L_2) + \bar{\delta}(L_2, v_k).$$

Using the 3-approximate distance oracle, we write:

$$\bar{\delta}(u, L_1) + \bar{\delta}(L_1, L_2) \leq 3 \bar{\delta}(u, L_2).$$

Therefore,

$$\delta(u, v_k) \leq 3 \bar{\delta}(u, L_2) + \bar{\delta}(L_2, v_k).$$

Rewriting the first term, we obtain:

$$\delta(u, v_k) \leq 2 \bar{\delta}(u, L_2) + \bar{\delta}(u, L_2) + \bar{\delta}(L_2, v_k).$$

Using the 3-approximation property again on $\bar{\delta}(L_2, v_k)$, we get:

$$\delta(u, v_k) \leq 2 \bar{\delta}(u, L_2) + 3 \bar{\delta}(u, v_k).$$

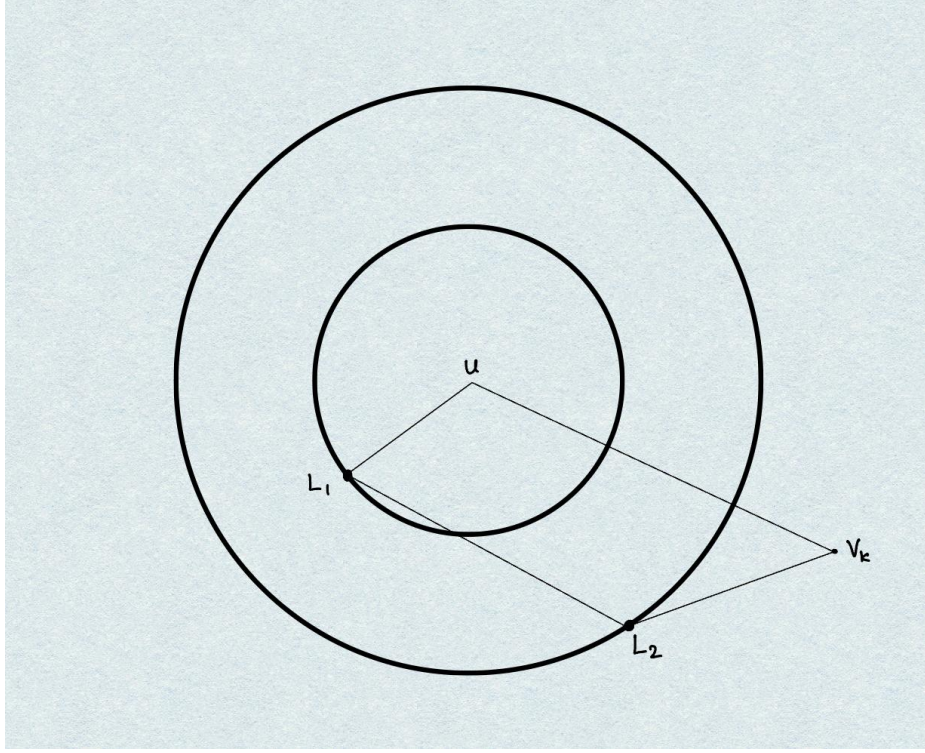
Since $v_k \notin \text{Ball}(u, V, L_2)$, we have

$$\bar{\delta}(u, L_2) \leq \bar{\delta}(u, v_k).$$

Substituting this, we obtain:

$$\delta(u, v_k) \leq 5 \bar{\delta}(u, v_k).$$

Thus, in this case, the reported distance is within a constant factor of the actual distance.



Case 3: $v_k \in \mathbf{Ball}(u, L_1, L_2) \setminus \mathbf{Ball}(u, V, L_1)$

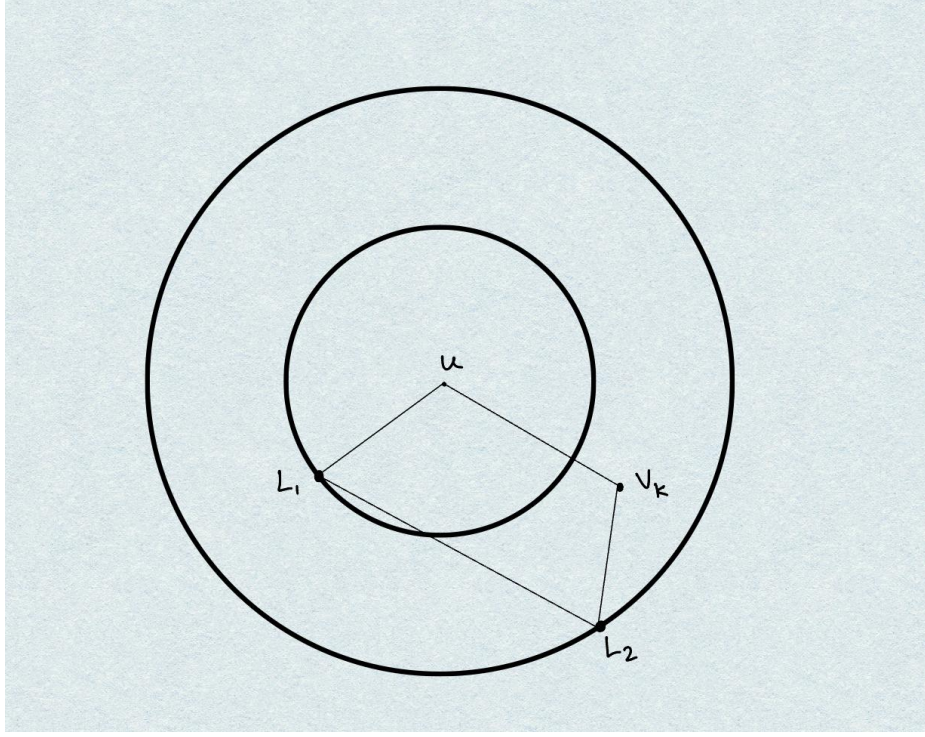
In this intermediate case, the queried vertex lies inside the outer ball but outside the inner ball. We attempt to report the distance by routing through the landmarks:

$$\delta(u, v_k) = \bar{\delta}(u, L_1) + \bar{\delta}(L_1, L_2) + \bar{\delta}(L_2, v_k).$$

To analyze this expression, we attempted to apply various inequalities using standard metric properties, including the triangle inequality and relations obtained by considering diagonals of the quadrilateral formed by the vertices u , L_1 , L_2 , and v_k . However, each such attempt failed to produce a bound that guarantees

$$\delta(u, v_k) \leq 5 \bar{\delta}(u, v_k).$$

In particular, the lack of a clear ordering between $\bar{\delta}(u, v_k)$ and $\bar{\delta}(u, L_2)$ prevents us from completing the analysis for this case.



We successfully analyzed all other cases but were unable to complete the analysis for vertices lying between the two ball boundaries. This highlights a limitation of the current method and suggests that additional structure or sampling constraints are required to achieve a strict 5-approximation guarantee.

Problem 3. Show that the coupon collector experiment gets over in $O(n \log n)$ time.

Let T be the number of draws. Let A_j denote the event that the j -th ball (where $j \in \{1, \dots, n\}$) is **not** chosen after T draws.

The probability of not picking a specific ball j in a single draw is $1 - \frac{1}{n}$. Since the draws are independent with replacement, the probability that ball j is not picked after T draws is:

$$P(A_j) = \left(1 - \frac{1}{n}\right)^T$$

We use the standard inequality $1 - x \leq e^{-x}$ (for all x). Setting $x = 1/n$:

$$P(A_j) \leq (e^{-1/n})^T = e^{-T/n}$$

We set the number of trials $T = c \cdot n \ln n$ for some constant $c > 1$. Substituting this into the inequality:

$$P(A_j) \leq e^{-\frac{cn \ln n}{n}} = e^{-c \ln n} = (e^{\ln n})^{-c} = n^{-c}$$

Bounding the error Probability: The experiment fails if there is **at least one** ball that has not been collected after T steps. This is the event $\bigcup_{j=1}^n A_j$.

$$\begin{aligned} P_{err} &= P\left(\bigcup_{j=1}^n A_j\right) \\ &\leq \sum_{j=1}^n P(A_j) \end{aligned}$$

Since $P(A_j)$ is the same for all n balls:

$$\begin{aligned} P_{err} &\leq n \cdot P(A_1) \\ &\leq n \cdot n^{-c} \\ &= n^{1-c} \end{aligned}$$

This shows that the probability that we end up collecting all the coupons is

$$1 - P_{err} \geq 1 - \frac{1}{n^{c-1}}$$

If we choose $c > 1$ (e.g., $c = 2$), the failure probability is $\frac{1}{n}$, which approaches 0 as $n \rightarrow \infty$. Thus, the Coupon Collector experiment finishes in $O(n \log n)$ steps with high probability. ■